In [ ]:
```
# Initialize Otter
import otter
grader = otter.Notebook("practice05.ipynb")
```

Student names and e-mails:

YOUR NAME — your@calvin.edu

YOUR NAME — your@calvin.edu

Practice 05 — Catching Spam with kNN

5,160 real text messages, each labeled **spam** (junk) or **ham** (a normal, wanted message — anything that is *not* spam). Can k-nearest neighbors learn to tell them apart — and when it's wrong, *which* way is it wrong?

SLO	Description
06A	Train and interpret k-NN classification models and explain how the choice of k affects overfitting and underfitting
06B	Compute and interpret classification metrics — accuracy, precision, recall, and the confusion matrix
06C	Split data into training and test sets to evaluate how well a model generalizes to unseen data

The Dataset: SMS Spam



Why "spam"? In a 1970 *Monty Python* sketch, a café serves Spam (the canned meat) with everything, and a table of Vikings keeps singing "Spam, Spam, Spam…" until nobody can hear anything else. Early internet users borrowed the word for junk messages that drown out the ones you want. (More here (https://en.wikipedia.org/wiki/Spam_(Monty_Python)) — culture matters.)

The messages come from the SMS Spam Collection (https://archive.ics.uci.edu/dataset/228/sms+spam+collection) (Almeida & Gómez Hidalgo, 2011; CC BY 4.0), gathered in the UK and Singapore around 2010 — expect lots of "txt", "u" and "£". Repeated messages were removed, so each message appears once.

Column	What it holds
`label`	**spam** (junk) or **ham** (not spam: a normal message from a friend, family, work…) — the answer the model learns from
`text`	the message itself

Setup (given — just run it)

In [ ]:
```
import numpy as np
import pandas as pd
import plotly.express as px
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, confusion_mat
rix

pd.set_option("display.max_colwidth", 200)   # show whole messages

sms = pd.read_csv("https://cs.calvin.edu/courses/data/202/26fa/datasets/spam.csv")
y = sms["label"] == "spam"          # the label: True = spam, False = ham
print(len(sms), "messages |", y.sum(), "spam")
sms.head()
```

From Messages to Numbers

kNN measures distances between rows of numbers. A text message is not a row of numbers — yet. One way to make it one: give every word its own column and count how often it appears. This is called a **bag of words** (word order is thrown away):

message	free	win	meeting	lunch	you
"Win a free trip!"	1	1	0	0	0
"Free lunch meeting tomorrow"	1	0	1	1	0
"Are you coming to the meeting?"	0	0	1	0	1

The problem: a word like "you" shows up in almost every message, so it says little about *this* one. **TF-IDF** reweights the counts:

- **TF** (term frequency) — how often the word appears in *this* message
- **IDF** (inverse document frequency) — how *rare* the word is across all the messages

A word that is frequent here but rare elsewhere ("win", "prize") gets a high weight; a word that is everywhere ("you") gets a low one:

message	free	win	meeting	lunch	you
"Win a free trip!"	0.6	0.7	0.0	0.0	0.0
"Free lunch meeting tomorrow"	0.4	0.0	0.3	0.3	0.0
"Are you coming to the meeting?"	0.0	0.0	0.2	0.0	0.4

scikit-learn's `TfidfVectorizer` does this for us. But first — the split.

Part 1 — Hold Some Messages Out (SLO 06C)

Task 06C.1 — Train/Test Split *(2 pts)*

Split the **messages** `sms["text"]` and the labels `y` into a training set and a test set, with `train_test_split` : **20%** for testing and `random_state=42` . Store the four pieces in `text_train, text_test, y_train, y_test` (same order as in class).

Also pass **`stratify=y`** . Only about 1 message in 8 is spam; `stratify` makes sure the training and test sets both keep that same share, instead of leaving it to luck.

In [ ]:
```
text_train, text_test, y_train, y_test = train_test_split(sms["text"], y, test_size=0.2,
stratify=y, random_state=42)  # SOLUTION
print(len(text_train), "training messages |", len(text_test), "test messages")
print("share of spam:", round(y_train.mean(), 3), "(training) |", round(y_test.mean(),
3), "(test)")
```

In [ ]:
```
grader.check("06C.1")
```

Turning Text into TF-IDF (given — just run it)

The vectorizer builds its word list — and learns how rare each word is — with `fit`. Wednesday's rule applies here too: **test messages never go into `fit`**. So:

- `fit_transform` on the **training** messages: learn the words, then turn those messages into numbers
- `transform` only on the **test** messages: use the words learned from training — the way a real filter meets tomorrow's messages

In [ ]:
```
vectorizer = TfidfVectorizer(
    stop_words="english",                    # drop very common English words ("the", "an
d", ...)
    min_df=2,                                # keep only words that appear in at least 2
messages
    token_pattern=r"(?u)\b[a-zA-Z]{2,}\b",   # a "word" = 2 or more letters, so no numbe
rs
)
X_train = vectorizer.fit_transform(text_train)   # fit: learns the words from the TRAINI
NG messages only
X_test = vectorizer.transform(text_test)         # the same columns, for the test messag
es

words = vectorizer.get_feature_names_out()
print(X_train.shape, X_test.shape, "->", len(words), "word columns")
```

Each message is now a row with one number per word — almost all of them 0. Here is one training message, with the words it contains and their weights:

In [ ]:
```
row = 2
print(text_train.iloc[row], "\n")
weights = pd.Series(X_train[row].toarray()[0], index=words)
weights[weights > 0].sort_values(ascending=False).round(2)
```

Could we do better? A **lemmatizer** would reduce each word to its base form ("offers" → "offer", "winning" → "win"), so the variants share one column. We'll stop here — look it up if you're curious.

Question 06C.2 *(1 pt)*. The vectorizer's word list came from the training messages only. What happens to a word that appears in a test message but never in the training messages? Why is that the honest way to test a spam filter?

Type your answer here, replacing this text.

Sample answer: The word has no column, so `transform` simply ignores it — the test message is described only by words the filter learned. That's exactly the situation of a real filter: it was built from past messages and meets new ones with new words (new scams, new slang). Fitting the vectorizer on the test messages too would let information from the "future" leak into the model, and the test score would be too optimistic.

Part 2 — A kNN Spam Filter (SLO 06A)

Task 06A.1 — Fit and Predict *(2 pts)*

Train a `KNeighborsClassifier` with **k = 5** on the training set, store it in `knn`, and store its predictions for the **test** messages in `y_pred`.

Two extra settings, for text: `metric="cosine"` and `algorithm="brute"`. Cosine distance compares *which words* two messages use, not how long they are — and it is fast on rows that are mostly zeros.

In [ ]:
```
knn = KNeighborsClassifier(n_neighbors=5, metric="cosine", algorithm="brute").fit(X_train, y_train)  # SOLUTION
y_pred = knn.predict(X_test)  # SOLUTION
print("messages called spam:", y_pred.sum(), "of", len(y_pred))
```

In [ ]:
```
grader.check("06A.1")
```

Part 3 — How Good Is the Filter? (SLO 06B)

> 🎯 **Predict First:** A lazy "filter" that lets every message through — it never says spam. What is its accuracy on the test set?

In [ ]:
```
lazy = np.zeros(len(y_test), dtype=bool)   # never says spam
print("lazy filter accuracy:", round(accuracy_score(y_test, lazy), 3))
```

Almost 88% — and it catches **no** spam at all. Same as Wednesday's Chinstraps: when a group is rare, accuracy alone hides the model's mistakes. Here, **spam** is the rare group we want to find (the "positive" class).

Task 06B.1 — Confusion Matrix and Metrics *(3 pts)*

Reminder: **ham** = a normal message, i.e. *not* spam. In `y`, spam is `True` (the positive class we want to catch) and ham is `False`.

For your k = 5 filter on the test set:

1. **The confusion matrix is given** — the first lines of the cell build it for you. `confusion_matrix(y_test, y_pred, labels=[False, True])` counts the test messages in a 2×2 array: **rows = true label, columns = predicted label**, both in the order `[False, True]`, i.e. ham first, then spam. Wrapping that array in a `pd.DataFrame` with `index` and `columns` set to `["ham", "spam"]` just puts readable names on the rows and columns, so you can look up a cell by name — e.g. `cm.loc["ham", "spam"]` = messages that were really ham but predicted spam:

	predicted **ham**	predicted **spam**
true ham	true negative — normal message kept	**false positive** — normal message sent to the spam folder
true spam	**false negative** — spam let through	true positive — spam caught
2. Store the test `accuracy`, `precision` and `recall` (for spam) — each a number between 0 and 1.

The plot at the end is given.

In [ ]:
```
# Given: rows = true label, columns = predicted label; order [False, True] = [ham, spam]
names = ["ham", "spam"]
cm = pd.DataFrame(confusion_matrix(y_test, y_pred, labels=[False, True]), index=names, c
olumns=names)
display(cm)

accuracy = accuracy_score(y_test, y_pred)  # SOLUTION
precision = precision_score(y_test, y_pred)  # SOLUTION
recall = recall_score(y_test, y_pred)  # SOLUTION
print(f"accuracy: {accuracy:.3f} | precision: {precision:.3f} | recall: {recall:.3f}")

px.imshow(cm, text_auto=True, color_continuous_scale="Blues",
          labels=dict(x="predicted", y="true", color="messages"),
          title="Confusion Matrix: 1,032 Test Messages, k = 5")
```

In [ ]:
```
grader.check("06B.1")
```

Look at the Mistakes (given — just run it)

The **false positives** (ham the filter called spam) and a few **false negatives** (spam it let through):

In [ ]:
```
results = pd.DataFrame({"text": text_test, "true": y_test.map({True: "spam", False: "ha
m"}),
                        "predicted": np.where(y_pred, "spam", "ham")})
print("FALSE POSITIVES — ham called spam:")
display(results[(results["true"] == "ham") & (results["predicted"] == "spam")])
print("FALSE NEGATIVES — spam let through (first 8):")
display(results[(results["true"] == "spam") & (results["predicted"] == "ham")].head(8))
```

Question 06B.2 *(1 pt)*. For a spam filter, which mistake costs more: a false positive or a false negative? So which metric would you watch most closely — precision or recall? How does your k = 5 filter do on it?

(Remember: false positives lower precision; false negatives lower recall.)

Type your answer here, replacing this text.

Sample answer: A false positive is worse: a real message — a job offer, a friend, the bank — disappears into the spam folder, and you may never see it. A false negative is just one annoying message in your inbox. So precision matters most. Our filter has precision 0.98 (2 of the 99 messages it called spam were ham) but recall only 0.76 (it let 31 of 128 spam messages through) — a cautious filter, which is what we want. (Someone else might answer recall, e.g. if spam is scams that hurt vulnerable users; the answer should say whose cost matters. And these are scores from one split.)

Part 4 — Choosing k: Overfitting and Underfitting (SLO 06A)

k = 5 was a guess. Monday's penguin plot compared accuracy on the data the model learned from with accuracy on new data. Let's make the same plot for the spam filter — with recall and precision, since accuracy hides the rare spam.

Task 06A.2 — Training vs. Test, for Many k *(3 pts)*

For every k in `ks`, fit a kNN filter (same `metric="cosine", algorithm="brute"`) on the training set. Then compute its **recall** and **precision** on the **training** set and on the **test** set. The loop collects them into `scores`; the plots are given.

In [ ]:
```
ks = list(range(1, 102, 4))       # k = 1, 5, 9, ..., 101
rows = []
for k in ks:
    # fit a kNN filter with k neighbors on the training set
    model = KNeighborsClassifier(n_neighbors=k, metric="cosine", algorithm="brute").fit
(X_train, y_train)  # SOLUTION
    # predict the training messages and the test messages
    pred_train = model.predict(X_train)  # SOLUTION
    pred_test = model.predict(X_test)  # SOLUTION
    # recall and precision, on each set
    recall_train = recall_score(y_train, pred_train)  # SOLUTION
    recall_test = recall_score(y_test, pred_test)  # SOLUTION
    precision_train = precision_score(y_train, pred_train)  # SOLUTION
    precision_test = precision_score(y_test, pred_test)  # SOLUTION
    rows.append({"k": k, "recall_train": recall_train, "recall_test": recall_test,
                 "precision_train": precision_train, "precision_test": precision_test})
scores = pd.DataFrame(rows)

px.line(scores, x="k", y=["recall_train", "recall_test"], markers=True,
        labels={"value": "recall", "variable": "scored on"}, title="Recall vs. k").show
()
px.line(scores, x="k", y=["precision_train", "precision_test"], markers=True,
        labels={"value": "precision", "variable": "scored on"}, title="Precision vs.
k").show()
scores.round(3).head()
```

In [ ]:
```
grader.check("06A.2")
```

Question 06A.3 *(1 pt)*. Where do you see **overfitting** in the plots, and where **underfitting**? Why does recall drop as k grows? (Think about how the vote goes when only 1 in 8 messages is spam.) Which k would you pick for your filter — and how much would you trust that choice?

Type your answer here, replacing this text.

Sample answer: Overfitting at k = 1: recall and precision on the training set are almost perfect (each message is its own nearest neighbor) but test recall is 0.82 — the gap is the sign of memorizing. Underfitting at large k: recall falls on both sets (test recall about 0.54 at k = 101), because with many neighbors a spam message is usually outvoted by the ham around it — the rare class gets drowned out, like Monday's Chinstraps at k = 100. Precision stays high throughout. Since we care most about precision, a small k around 5 (test precision 0.98, recall 0.76) is a good choice; k = 1 finds more spam but with more false positives. But all of this comes from one split of 1,032 test messages (only 128 spam) — another split could move these numbers by several points, so the exact best k is uncertain. Week 7's cross-validation addresses this.

Bonus: Which Words Scream "Spam"? (given — just run it)

The average TF-IDF weight of each word, in spam and in ham training messages. These are the words kNN's distances lean on — but always together with the rest of the message ("Are you **free** Saturday?" is not spam).

In [ ]:
```
spam_words = pd.Series(np.asarray(X_train[y_train.values].mean(axis=0))[0], index=words)
ham_words = pd.Series(np.asarray(X_train[~y_train.values].mean(axis=0))[0], index=words)
pd.DataFrame({"top spam words": spam_words.sort_values(ascending=False).head(15).index,
              "top ham words": ham_words.sort_values(ascending=False).head(15).index})
```

Submission

Save your notebook, then upload the **`.ipynb` file** directly to our Moodle assignment page. Don't export, zip, or submit anything else — just the notebook.